UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros de Telecomunicación
Grupo de Tratamiento de Imágenes

Proyectos en Ingeniería de Datos y Sistemas (PIDS)

Proyecto 2

Sistema de detección de vehículos y conducción autónoma

MEMORIA DEL PROYECTO FASES 1 Y 2

Grupo	2
Autores	Javier Saguar Alejandro Cuevas Mónica Fernández Pedro José Orrego Daniel Naval
Entrega	Entrega 2 · Bloques 1 y 2
Fecha de entrega	25/10/2026

Reparto del trabajo

Integrante	Tareas del Bloque 1	Tareas del Bloque 2
Javier Saguar	Lote A (vídeos 01-02): extracción, anotación y revisión cruzada. Fichas de los vídeos, criterios de anotación y depuración del dataset. Aumento de datos sobre el script de Alejandro, que mejoró y ejecutó. Redacción de la memoria.	Cadena de entrenamientos (siete arquitecturas y tres tasas de aprendizaje), evaluación final en test y comprobación con 40 épocas. Gráficas comparativas de la web y de la memoria.
Alejandro Cuevas	Lote B (vídeos 03-04): extracción, anotación y revisión cruzada. Análisis exploratorio de los datos (EDA) y script de entrenamiento sobre el que se integró el aumento. Redacción de la memoria.	Adaptación del código de entrenamiento y test, propuestas de mejora del entrenamiento y redacción de la memoria del Bloque 2.
Mónica Fernández	Lote C (vídeos 05-06): extracción, anotación y revisión cruzada. Calibración entre anotadores y revisión de la memoria.	Correcciones memoria Bloque 1.
Pedro José Orrego	Lote D (vídeos 07-08): extracción, anotación y revisión cruzada. Calibración, selección de figuras del EDA y diseño de la web de figuras interactivas (GitHub Pages).	Web interactiva del Bloque 2 y navegación común con la del Bloque 1 (GitHub Pages).
Daniel Naval	Lote E (vídeos 09-10): extracción, anotación y revisión cruzada. Calibración y redacción de la memoria.	

Tabla 1. Reparto del trabajo de los Bloques 1 y 2 entre los cinco integrantes.

Índice

Reparto del trabajo 2
Índice 3
1. Introducción 4
2. Recolección de datos 4
3. Anotación de datos 5
4. Análisis exploratorio de los datos 6
5. Aumento de datos 11
6. Depuración de datos 13
7. El pipeline de entrenamiento 14
8. Modelos y transfer learning 15
9. Hiperparámetros 15
10. Métricas 17
11. Experimentos 17
12. Análisis cualitativo 22
Referencias 24
Anexos 25
Respuesta a la revisión anterior 30

1. Introducción

1.1. Contexto y motivación

El proyecto consiste en desarrollar un detector de robots RoboMaster rivales a partir de las imágenes captadas por la cámara de un robot móvil. Esta tarea presenta varias dificultades, ya que el movimiento del robot, los cambios de tamaño aparente, las condiciones de iluminación y la presencia de obstáculos pueden afectar a la localización de los robots. La detección obtenida servirá como base para desarrollar posteriormente una estrategia de apuntado y actuación.

Los robots que hay que detectar son RoboMaster EP de DJI [19], el mismo modelo que lleva la cámara. Es un robot terrestre educativo con chasis de ruedas mecanum, que le permiten desplazarse en cualquier dirección, y una torreta (gimbal) que orienta la cámara y el lanzador (blaster). Las placas de impacto del chasis y la torreta llevan indicadores LED de color (Figura 1). Esa silueta, con sus luces, es lo que el detector tiene que reconocer desde cualquier ángulo y distancia.



Figura 1. Partes de un RoboMaster EP en un fotograma de V03. Las antenas quedan fuera de la caja (Tabla 4).

1.2. Objetivos del proyecto

El trabajo se divide en tres bloques. El primero se centra en la creación y el análisis del conjunto de datos, el segundo en el entrenamiento y ajuste del detector, y el tercero en su integración con el robot. En este primer bloque se obtuvieron 808 imágenes originales anotadas, de las cuales finalmente se seleccionaron 796 tras revisar las etiquetas y comprobar su calidad.

2. Recolección de datos

2.1. Extracción de imágenes

La cámara del robot graba a 1280 × 720 píxeles y unos 30 fps, así que dos fotogramas seguidos son casi iguales. Por eso se extrae uno de cada 24 fotogramas, una imagen cada 0,79 segundos. Con los 19 508 fotogramas de los diez vídeos (643,8 s de grabación) se obtienen 808 imágenes: cada vídeo aporta sus fotogramas divididos entre 24, redondeando hacia abajo (Tabla 2).

Vídeo	Escenario	Duración s	Frames	Imágenes
V01	Vestíbulo ETSIT	65,4	1982	82
V02	Aula A-201.L	59,9	1815	75
V03	Aula A-201.L	60,2	1823	75

Vídeo	Escenario	Duración s	Frames	Imágenes
V04	Exterior edificio A	66,1	2003	83
V05	Pasillo A-201.L	59,6	1807	75
V06	Vestíbulo ETSIT (mesas)	74,1	2244	93
V07	Vestíbulo ETSIT	61,2	1855	77
V08	Vestíbulo ETSIT (mesas)	81,2	2460	102
V09	Exterior edificio A	56,2	1704	71
V10	Aula A-201.L	59,9	1815	75
Total	1280 × 720 a 30,30 fps	643,8	19 508	808

Tabla 2. Grabaciones propias y número exacto de imágenes extraídas.

El intervalo se eligió por el compromiso entre redundancia y cantidad de datos. Para comprobarlo, se repitió la extracción con otros intervalos y se midió cuánto cambia cada imagen respecto a la anterior, con la misma medida del EDA: la diferencia media de gris entre miniaturas, con la que una pareja por debajo de 5 se considera casi duplicada (Tabla 3).

Intervalo (fotogramas)	Segundos	Imágenes	Casi duplicadas	Diferencia mediana	Robots que anotar
6	0,20	3248	11,1 %	15,0	≈ 5 710
12	0,40	1622	4,6 %	22,6	≈ 2 850
24	0,79	808	1,4 %	29,2	1 420
48	1,58	400	0,3 %	32,8	≈ 700
96	3,17	196	0,0 %	36,9	≈ 340

Tabla 3. Redundancia de las imágenes extraídas según el intervalo, sobre los diez vídeos. Los robots se estiman con la media de 1,76 por imagen.

Con 12 fotogramas, el 4,6 % de las imágenes son casi duplicadas y habría que anotar el doble de robots para añadir poca información (diferencia mediana de 22,6). Con 24 la redundancia cae al 1,4 %, y duplicar el intervalo apenas cambia la diferencia (de 29,2 a 32,8) pero deja la mitad de ejemplos, 400 imágenes. Extraer más fotogramas tampoco aumentaría la variedad: la imagen más parecida a cada una ya es del mismo vídeo en el 74 % de los casos (apartado 4.2). La variedad depende de grabar escenarios distintos.

3. Anotación de datos

Se utilizó labelImg [1], que guarda las cajas en el formato XML de PASCAL VOC [20], con una única clase robot. Cada integrante anotó su lote y otro revisó las cajas. Se conservaron imágenes negativas con XML vacío para que el entrenamiento no las ignore.

Caso	Criterio operativo
Robot rival	Incluir chasis, ruedas, gimbal y cañón; excluir antenas.
Elementos ajenos	No anotar el cañón propio, reflejos ni robots en pantallas.
Oclusión y borde	Anotar si es reconocible y se ve al menos un 30 %; caja sobre la parte visible.
Difficult	Marcar si tiene menos de 30 px de ancho, menos de la mitad visible o está muy movido.
Robot muy pequeño	No anotar por debajo de 10 px de lado; conservar criterios en el aumento.
Negativo	Guardar imagen y XML sin objetos.

Tabla 4. Criterios comunes aplicados a los lotes.

Como se observa en la figura siguiente, el criterio distingue cajas normales y difficult sin crear una segunda clase. Una frontera superior distinta alteraría tamaños y centros, por lo que se excluyen antenas y se ajusta el resto del robot de manera consistente.



Figura 2. Anotaciones de interior y exterior. Azul: normal; naranja: difficult.

Los cinco anotadores etiquetaron las mismas 20 imágenes de calibración. El IoU medio entre parejas fue 0,81; el borde superior presenta la mayor discrepancia, 7 px de mediana frente a 4–5 px en los otros bordes. Esto se debe a que la parte superior del robot es la más ambigua de anotar, debido a las antenas.

4. Análisis exploratorio de los datos

4.1. Inventario y equilibrio

El EDA se calcula sobre los 808 originales, antes de excluir imágenes. Los aumentos no se mezclan en estas estadísticas. Las cifras de la tabla y la figura permiten separar cantidad de imágenes, número de objetos y dificultad. El notebook del EDA adjunto [7] contiene más análisis (acuerdo entre anotadores, diversidad visual, duplicados y línea temporal).

Estadística	Valor
Secuencias / imágenes	10 / 808
Imágenes medias por secuencia	80,8

Estadística	Valor
Cajas / cajas medias por imagen	1420 / 1,76
Difficult	375 cajas · 26,4 %
Imágenes negativas	41 · 5,07 %
Área media de caja	33,627 miles de píxeles²
Área mínima / máxima	1,020 / 661,074 miles de píxeles²
Aspecto mediano ancho / alto	1,11

Tabla 5. Estadísticas de los originales anotados.

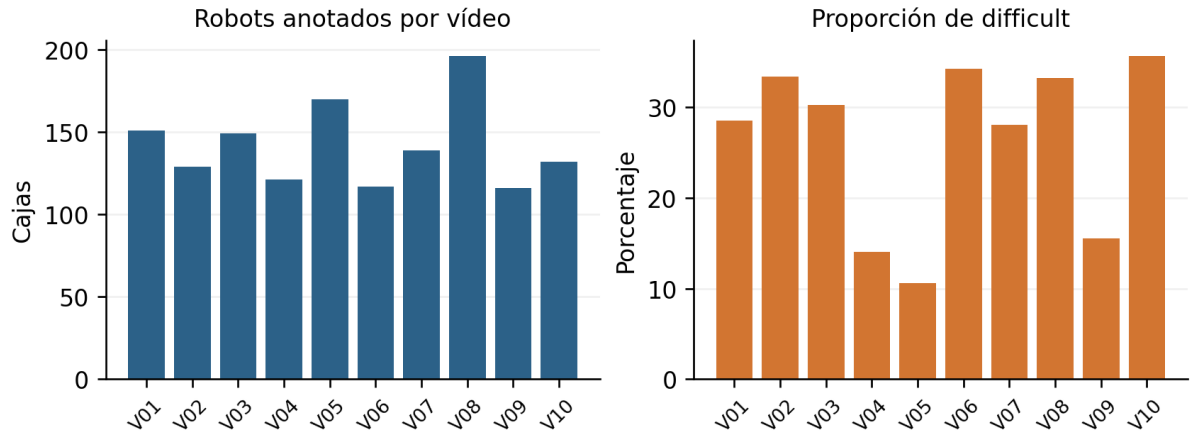


Figura 3. Recuento y dificultad por vídeo; sus diferencias motivan analizar cada secuencia.

No hemos intentado que todos los conjuntos sean exactamente iguales. Por ejemplo, V05 registra 170 robots, mientras que V06 contiene 117. Además, V06 incluye 22 ejemplos negativos, mientras que V08 no tiene ninguno y tiene 65 casos especialmente difíciles. Se conservan porque son situaciones reales, pero obligan a medir el AP de los difficult por separado (apartado 10): un AP global alto podría esconder que fallan.

4.2. Tamaño, forma e iluminación

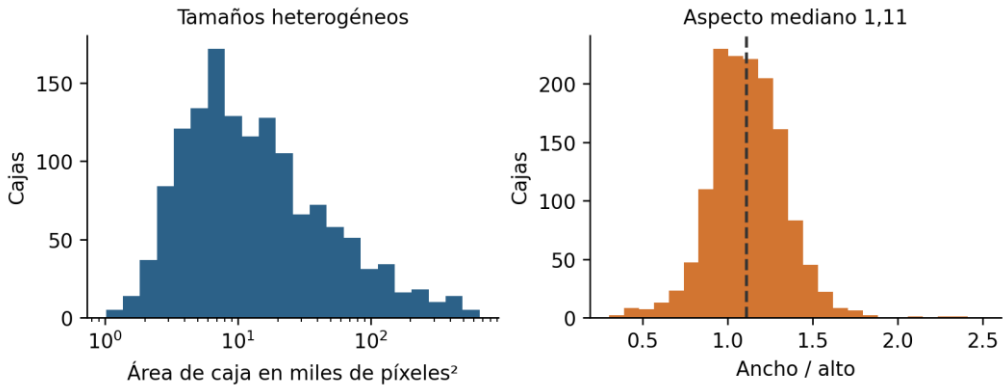


Figura 4. Área y forma de las cajas de los 1420 robots. El eje de área es logarítmico.

La distribución de la figura tiene una cola amplia de áreas, pero no hay cajas originales de menos de 30 px de ancho. La relación de aspecto mediana es 1,11. El análisis de cobertura de anclas obtiene un

99,6 % con IoU ≥0,5: no se cambia su configuración por ahora y se evita deformar las proporciones durante el aumento.

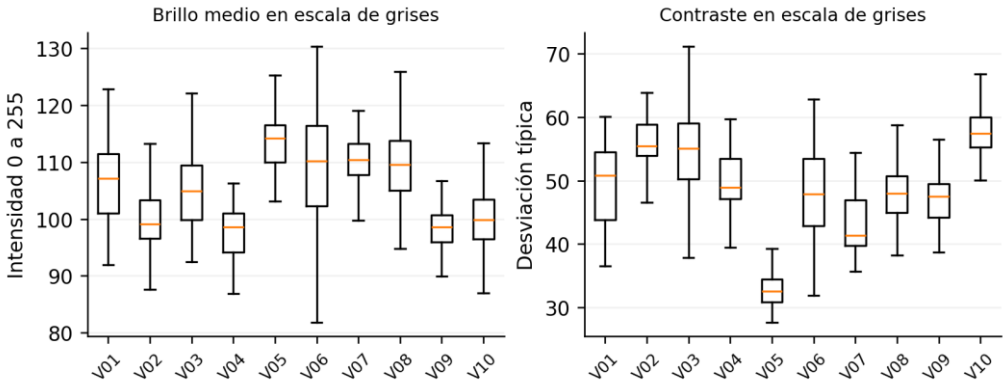


Figura 5. Brillo y contraste por secuencia, calculados en escala de grises.

Los canales RGB reflejan el color predominante de cada imagen y no solo las condiciones de iluminación, ya que también están influidos por elementos como el suelo o las paredes. Por ello, se complementan con medidas de intensidad y nitidez. Además, el tono azulado habitual en exteriores justifica aplicar ajustes de color moderados para no confundirlo con una iluminación deficiente. Se identificaron 52 imágenes poco nítidas para revisión visual; en cada vídeo, las menos nítidas tienen más cajas difficult (55 % frente a 11 %, Figura 21, Anexo C).

La miniatura más parecida a una imagen procede del mismo vídeo en el 74 % de los casos. Esa correlación aconseja separar grabaciones completas, evitando una validación artificialmente fácil con fotogramas vecinos.

4.3. Posición y evolución temporal

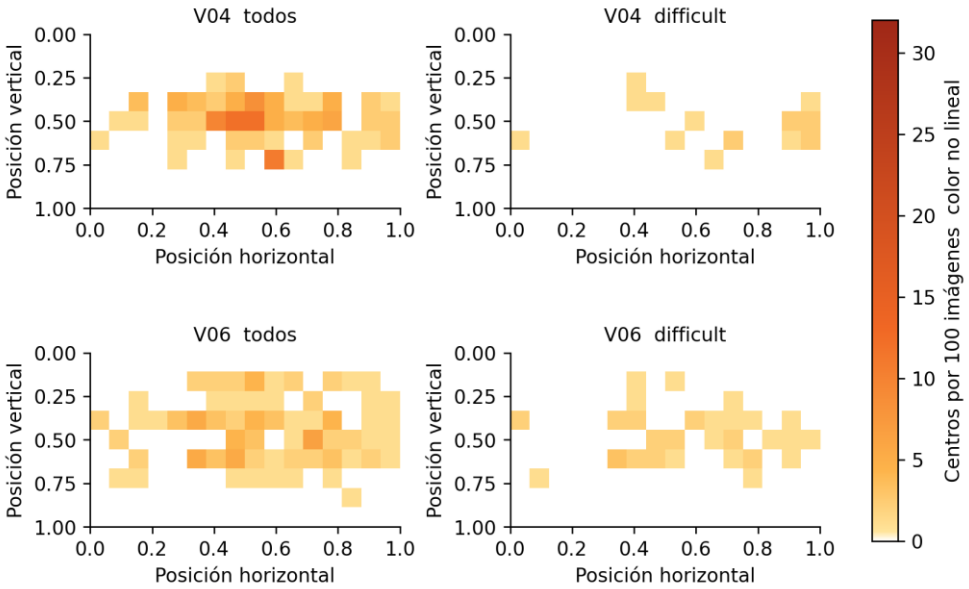


Figura 6. Centros de las cajas en V04 (exterior) y V06 (interior). Todos y difficult comparten escala. Versión interactiva de los diez vídeos en el Anexo A: centros y evolución.

La figura compara la posición de las anotaciones en el exterior (V04) y el interior (V06) con muchos negativos. Se normalizan los centros por tamaño de imagen y el histograma 16 × 9 por número de imágenes. Aunque la mayoría de cajas están cerca del centro, la proporción de cajas cercanas a los

bordes es mayor en las difficult, porque muchas son robots cortados por el borde o que entran y salen de la imagen. Los robots aparecen siempre en una franja a media altura y de forma simétrica: el volteo horizontal es válido y el desplazamiento vertical del aumento debe ser pequeño (Tabla 8).

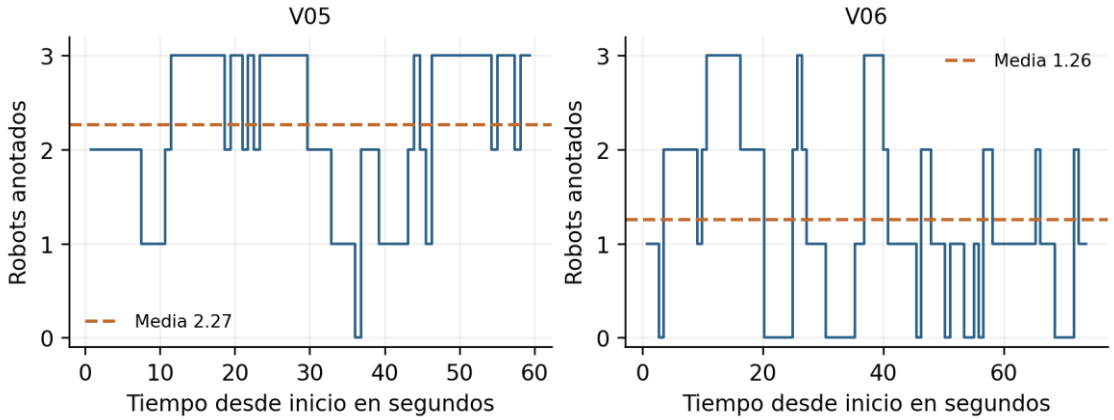


Figura 7. Robots por fotograma anotado en V05 y V06, cada 24 fotogramas, con la media de cada vídeo. Versión interactiva en el Anexo A.

Los recuentos muestran qué tramos de cada vídeo tienen más y menos robots y también permiten encontrar discrepancias en las anotaciones cuando los robots se mueven. En torno al segundo 20 de V05, por ejemplo, hay tres robots en pantalla, pero en algunos fotogramas no se anotaron, probablemente porque estaban borrosos o tapados. Para el entrenamiento, un robot visible sin caja enseña al detector que es fondo; por eso el criterio marca como difficult los robots movidos en lugar de dejarlos sin anotar (Tabla 4).

4.4. Tamaño de las cajas frente a su posición

Los mapas anteriores solo usan el centro de cada caja, pero el tamaño también depende de la posición. La cámara va a ras de suelo, así que un robot más cercano aparece más abajo y más grande (Figura 8, izquierda). La correlación entre la base de la caja y el logaritmo de su alto es de 0,68 (0,66 con el ancho): con la base por encima de la fila 360, la caja mediana mide 79 × 71 px; entre las filas 360 y 480, 121 × 107 px; y por debajo, 283 × 252 px. Las 15 cajas que más se apartan de esta relación se revisaron a ojo y son correctas: corresponden a momentos en que la cámara se inclina.

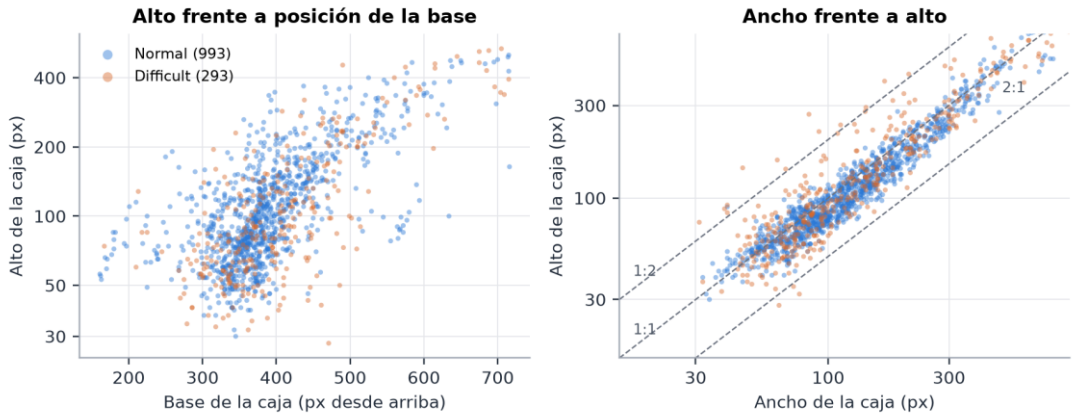


Figura 8. Izquierda: alto de la caja frente a la posición de su base. Derecha: ancho frente a alto de las 1420 cajas.

El ancho y el alto varían juntos (correlación de 0,94 entre sus logaritmos, Figura 8, derecha): las cajas son casi cuadradas en todo el rango de tamaños y quedan entre las proporciones 1:2 y 2:1 de las anclas

del detector. Las difficult tienen un tamaño mediano parecido (105 × 107 px frente a 114 × 98 px), pero una forma más variable: el 10 % más estrecho baja de 0,69 de ancho entre alto, frente a 0,91 en las normales, porque son robots cortados o movidos. De las 134 cajas que tocan el borde, el 61 % son difficult, frente al 23 % del resto.

Para el modelo, posición y tamaño no son independientes: subir o bajar un robot sin cambiar su tamaño crearía escenas imposibles, por eso el desplazamiento vertical se limita al ±5 % y la escala no deforma (Tabla 8). Las anclas por defecto ya cubren las formas observadas y no hace falta cambiarlas. Además, se midió el IoU entre robots de una misma imagen, porque el detector elimina con supresión de no máximos (NMS) las cajas que se solapan más de 0,5 con otra de mayor confianza. De 809 parejas, solo 92 se tocan y el IoU máximo es 0,26: la NMS no puede borrar robots reales que estén juntos.

4.5. Nuestros vídeos frente a validación y test

Validación y test son secuencias de otros grupos (apartado 7), grabadas con otras cámaras y en otros escenarios. Para saber en qué se parecen a nuestros datos se calcularon las mismas medidas en los tres conjuntos (Tabla 6; las distribuciones completas están en la Figura 18, Anexo C). Las de test se calcularon después de la evaluación final y no se usaron para tomar ninguna decisión.

Medida	Nuestros vídeos	Validación	Test
Secuencias / imágenes / robots	10 / 808 / 1420	11 / 672 / 1561	13 / 718 / 1721
Robots por imagen	1,76	2,32	2,40
Robots difficult	26,4 %	16,3 %	15,7 %
Cajas de menos de 30 / 50 px de ancho	0,0 / 3,9 %	2,5 / 11,3 %	0,5 / 9,1 %
Centro vertical, percentil 90	0,58	0,61	0,71
Brillo mediano (0-255)	104,8	97,8	103,9
Imágenes con $\lvert B - R \rvert > 12$	26,4 %	19,9 %	1,1 %

Tabla 6. Medidas de nuestros vídeos, validación y test con las mismas etiquetas y el mismo cálculo. B − R es la media del canal azul menos la del rojo de cada imagen.

Tres diferencias afectan al entrenamiento. La primera son los robots lejanos: en validación, el 11,3 % de las cajas mide menos de 50 px de ancho y el 2,5 %, menos de 30 px, frente al 3,9 % y ninguna en nuestros vídeos. Por eso el aumento aleja la imagen hasta ×0,6, y era previsible que fueran un punto débil: en el Bloque 2, el AP de objetos pequeños se queda entre 0,12 y 0,19 (apartado 11.2). La segunda es el color: una secuencia de validación tiene un tono rojizo (B − R = −28,0) que no aparece en nuestros vídeos, entre −4,0 y 8,3 en interior. Es un balance de blancos distinto, lo que simula el desplazamiento RGB (apartado 5.2). La tercera es que test tiene más robots en la parte baja de la imagen (percentil 90 del centro en 0,71 frente a 0,58), más cercanos y grandes, que la escala hasta ×1,25 cubre en parte. El brillo, en cambio, es parecido.

4.6. Consecuencias para el entrenamiento

La Tabla 7 resume qué decisión se tomó a partir de cada análisis y qué se observó después en el Bloque 2. El EDA anticipó los dos puntos débiles del detector, los difficult y los robots lejanos, que siguen siendo los errores principales con cualquier arquitectura (apartado 11.3).

Resultado del EDA	Decisión	Qué se observó en el Bloque 2
Imagen más parecida del mismo vídeo en el 74 % de los casos (4.2).	Validar y probar con secuencias de otros grupos.	AP@0,5 de 0,9513 en validación y 0,9523 en test.
26,4 % de difficult, junto a los bordes y en algunos vídeos (4.1, 4.4).	Conservarlos y medir su AP aparte.	Error principal: 74 de los 115 robots perdidos en test; AP@0,5 de 0,69 frente a 0,97.
Ninguna caja de menos de 30 px; en validación, un 2,5 % (4.5).	Alejar hasta ×0,6; probar 1024 px.	AP de objetos pequeños de 0,12 a 0,19; SSDlite320 (320 px) baja a 0,68.
Cajas casi cuadradas; anclas con IoU ≥ 0,5 para el 99,6 % (4.2, 4.4).	Anclas por defecto; escala sin deformar.	Los cuatro modelos con ResNet-50 superan 0,93 de AP@0,5.
IoU máximo de 0,26 entre robots (4.4).	NMS con umbral 0,5.	No borra robots reales; las cajas duplicadas (12) invitan a probar 0,3.

Tabla 7. Consecuencias del EDA para el entrenamiento y lo que se observó después en el Bloque 2.

5. Aumento de datos

El aumento se aplica solo al conjunto de entrenamiento y siempre a partir de los originales, nunca sobre copias ya aumentadas. Cada una de las 796 imágenes depuradas recibe tres copias, lo que da 3184 imágenes y 5566 cajas. Validación (672 imágenes) y test (718) son secuencias de otros grupos y se copian sin transformar, porque deben parecerse a lo que verá el robot y no a nuestras transformaciones.

El criterio para elegir cada transformación es doble, debe simular una variación que el robot puede encontrar en la demo y que nuestros diez vídeos no cubren del todo, y no debe producir escenas imposibles ni cajas que dejen de ajustarse al robot. Cada copia recibe una combinación distinta, con una media de 2,7 transformaciones, y la intensidad de cada una se sortea dentro de su rango.

5.1. Transformaciones geométricas

Transformación	Rango	p	Intención
Volteo horizontal	—	0,5	Un robot visto en espejo sigue siendo un robot válido y el EDA muestra que los centros de las cajas son simétricos en horizontal. Duplica las orientaciones sin crear escenas irreales.
Escala sin deformar	×0,6–1,25	0,7 (conjunta)	Alejar produce robots más pequeños y simula mayor distancia: no había cajas originales de menos de 30 px de ancho y la demo puede exigir detectar más lejos. Acercar produce robots cortados por el borde, frecuentes en nuestros vídeos.
Desplazamiento	±8 % horizontal, ±5 % vertical	0,7 (conjunta)	Cambia la posición del robot sin sacarlo de la franja a media altura en la que aparece siempre.
Giro	±6°	0,7 (conjunta)	La cámara se inclina al acelerar y en V06; el EDA atribuye a esa inclinación las cajas atípicas. Con giros mayores la caja deja de ajustarse al robot.

Tabla 8. Transformaciones geométricas utilizadas y su intención. Escala, desplazamiento y giro forman una sola transformación afín, que se aplica con p = 0,7 y sortea los tres valores a la vez.

Se aplican dos decisiones de implementación para evitar errores silenciosos. El hueco que deja un giro o un alejamiento se rellena de gris y no con un reflejo de la imagen, que podría duplicar un robot en el borde sin su caja y enseñar a la red a tratarlo como fondo. Al girar, la caja se recalcula aproximando el robot por una elipse inscrita en ella, lo que la mantiene ajustada; el método por esquinas la agranda (IoU de 0,85 con ±6°).

5.2. Color e iluminación

Transformación	Rango	p	Intención
Brillo y contraste	±20 %	0,25	Cubre iluminaciones no grabadas: otra hora del día, luces apagadas o contraluz.
Gamma	75–125	0,25	Complementa al brillo: aclara u oscurece las sombras sin saturar las zonas iluminadas.
Tono y saturación	±8 de tono, ±20 de saturación, ±10 de valor	0,15	Cambios suaves, como recomienda el EDA por el tono azulado de los exteriores. Un cambio fuerte alteraría el color de los LED, que es la pista más clara para reconocer el robot.
Desplazamiento RGB	±12 por canal	0,15	Simula el balance de blancos de otra cámara: una secuencia de validación tiene un tono rojizo (B − R = −28) que no hay en nuestros vídeos (apartado 4.5).

Tabla 9. Transformaciones de color e iluminación utilizadas y su intención. Se aplica una de las dos primeras con p = 0,5 y una de las dos últimas con p = 0,3.

El desplazamiento RGB suma una constante a cada canal, como un balance de blancos distinto, sin depender del contenido de la escena. Se mantiene del aumentador de la asignatura, pero con menos amplitud (±12 frente a ±15) y como alternativa al cambio de tono, para no acumular dos cambios de color en la misma copia ni alterar el color de los LED. La comparación con validación confirma que hace falta: con R +12 y B −12, una imagen de nuestros interiores pasa de B − R ≈ 0 a −24, cerca de los −28 de la secuencia rojiza (Figura 18, Anexo C).

5.3. Calidad de imagen

De estas cuatro transformaciones se aplica como mucho una por copia (p = 0,3), para no acumular degradaciones que harían la imagen peor que cualquier fotograma real. El desenfoque de movimiento se sortea aparte. Si a una copia no le toca ninguna transformación, se repite el sorteo, de modo que la probabilidad real de cada una es algo mayor que la indicada.

Transformación	Rango	p	Intención
Desenfoque de movimiento	Núcleo de 3–9 px	0,15	Es el defecto real de nuestros vídeos, porque se mueven la cámara y los robots. Sustituye al desenfoque uniforme del aumentador de la asignatura.
Compresión JPEG	Calidad 35–75	0,075	El vídeo llega comprimido desde el robot y pierde calidad cuando la señal es mala.

Transformación	Rango	p	Intención
Ruido de sensor	Gaussiano (varianza 10–40) o ISO	0,15 (0,075 cada tipo)	Reproduce el ruido que aparece con poca luz.
Menor resolución	×0,5–0,75	0,075	El SDK puede entregar el vídeo a menos resolución que la aplicación del robot.

Tabla 10. Transformaciones de calidad de imagen utilizadas y su intención. Compresión, ruido y menor resolución se sortean juntas: se aplica una de las cuatro con p = 0,3.

5.4. Transformación propia: oclusión sintética

En la demo los robots estarán detrás de obstáculos fijos, y en nuestros vídeos ya aparecen detrás de patas de silla y de piernas. Para entrenar con más casos parcialmente tapados se dibujan una o dos barras oscuras casi verticales en el 15 % de las copias. Ninguna barra puede tapar más del 35 % del ancho de un robot, de modo que sigue siendo reconocible y su caja no cambia, como establecen los criterios de anotación para obstáculos finos (Tabla 4).

Se prefiere a borrar rectángulos al azar (CoarseDropout en Albumentations [2]), que puede tapar un robot entero dejando su caja sobre el fondo. Es la transformación menos realista del conjunto, ya que una barra lisa no es una pata de silla; por eso se deja preparada una variante con postes texturados (apartado 6.1).

5.5. Ejemplo y transformaciones descartadas

La Figura 19 (Anexo C) aplica cada transformación por separado a una imagen de V06, con una intensidad fija para que se aprecie. En el aumento real se combinan y su intensidad se sortea dentro del rango de las tablas.

Se descartaron ocho transformaciones (Anexo B). Las principales son el volteo vertical y los giros de 90°, porque el robot nunca aparece boca abajo ni de lado; la perspectiva, porque en Albumentations 1.4.18 devuelve cajas que no coinciden con el robot; y el recorte 4:3 del aumentador de la asignatura, que ensancha los robots un 33 % y pierde el 14,1 % de las cajas.

La ejecución completa del aumento de datos está disponible en [8].

6. Depuración de datos

Se revisaron visualmente 69 imágenes para comprobar su calidad y utilidad. De ellas, 52 presentaban poca nitidez, 11 eran casi duplicadas y 6 planteaban dudas. Al final, descartamos una imagen en la que aparecía un robot que no podía anotarse de forma consistente y se eliminaron otras 11 por ser redundantes. En cambio, se mantuvieron 41 imágenes movidas con sus robots marcados como *difficult*, así como los negativos difíciles. Esta decisión se tomó porque una imagen poco nítida no tiene por qué ser inútil para el entrenamiento, pues también permite representar las condiciones reales en las que trabajará el detector.

Conjunto	Imágenes	Cajas
Original anotado	808	1420
Original tras depuración	796	1399

Conjunto	Imágenes	Cajas
Copias del aumento base	2388	4167
Entrenamiento total	3184	5566

Tabla 11. Efecto de la depuración y del aumento base.

6.1. Ampliaciones preparadas y límites

Se prepararon, además, copias aumentadas con sombras y llovizna sintética en los vídeos (V04 y V09), obstáculos con textura y más peso de las imágenes con difficult, para que el modelo aprendiera con degradaciones visuales y fuera más robusto. Al entrenar con este perfil, el AP@0,5 en validación apenas cambió (0,9426 frente a 0,9433 con el aumento base), sin diferencia en difficult (0,6471 frente a 0,6462). Una posible explicación es que el aumento base ya cubría parte de esa variabilidad, pero no se ha comprobado por falta de explicabilidad su causa raíz.

7. El pipeline de entrenamiento

El entrenamiento se realiza con PyTorch [18] y torchvision [4] utilizando 3184 imágenes (las 796 originales y tres versiones aumentadas de cada una). La validación incluye 672 imágenes de 11 secuencias grabadas, con 1561 robots y 255 difficult. Por su parte, el test contiene 718 imágenes de 13 secuencias externas, con 1721 robots y 270 difficult. Este conjunto se reserva para evaluar cada modelo una sola vez, sin utilizar sus resultados para tomar decisiones (apartado 11.6).

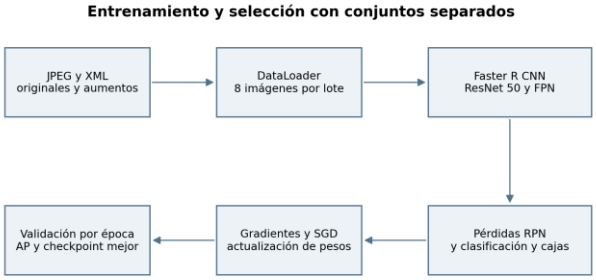


Figura 9. Flujo usado para ajustar pesos y seleccionar el checkpoint con validación.

La Figura 9 muestra el flujo de entrenamiento. Los JPEG y XML, tanto originales como aumentados, pasan al conjunto de datos, que carga cada muestra como una imagen RGB en formato tensor con valores entre 0 y 1 y un diccionario con sus cajas y etiquetas. El generador de datos agrupa las imágenes en batches de 8 y se las entrega al modelo, que las normaliza y escala internamente. Durante el entrenamiento, el modelo devuelve las pérdidas, su clasificación y regresión, cuya suma constituye la función de pérdida. Después, la retropropagación calcula los gradientes y SGD los utiliza para actualizar los pesos.

Un batch es el grupo de imágenes procesadas en cada paso, mientras que una época corresponde a una pasada completa por el conjunto de entrenamiento. Se trabaja por batches porque las 3184 imágenes no caben a la vez en la memoria de la GPU y porque así se actualizan los pesos varias veces por época.

Al terminar cada época, se evalúa el modelo con las 672 imágenes de validación, procedentes de secuencias que no ha visto, sin calcular gradientes ni actualizar los pesos. Se calculan la pérdida de

validación y el AP para medir su capacidad de generalización y elegir el mejor checkpoint. Es importante que esta fase no modifique los pesos para mantener una evaluación independiente. Al final de cada época se guarda el último checkpoint y, si mejora el AP@0,5, también el mejor.

Las cajas difficult también participan en el entrenamiento: antes de entregarlas al detector solo se quita su marca de difficult, no las cajas.

8. Modelos y transfer learning

Un detector está formado por un backbone, que es la red convolucional (ResNet-50 [11], MobileNetV3 [13] o VGG16 [14]) encargada de extraer características de la imagen, y el resto de componentes, que se ocupan de localizar y clasificar los objetos. Entre ellos están el cuello FPN [12], que combina la cabeza propuesta, la clasificación y su regresión. Se distingue el backbone porque es el componente que se puede intercambiar entre modelos, influye mucho en el coste computacional y cuenta con pesos preentrenados.

Todos los modelos utilizan transfer learning, es decir, parten de pesos aprendidos con COCO [15], una base de datos general con más de cien mil imágenes anotadas, y sustituyen la cabeza de clasificación por otra con dos clases: fondo y robot. De esta forma, la red aprovecha lo aprendido sobre bordes, texturas y formas, y adapta sus predicciones al objeto que queremos detectar [3,4]. Esto resulta especialmente importante porque solo disponemos de 796 imágenes originales de diez vídeos, una cantidad insuficiente para entrenar desde cero redes con decenas de millones de parámetros. Además, permite alcanzar el mejor checkpoint en pocas épocas (apartado 11.3).

Modelo	Tipo	Cómo funciona	Backbone y cuello
Faster R-CNN v1	Dos etapas	Primero propone regiones con posibles objetos (RPN) y después las clasifica y ajusta sus cajas.	ResNet-50 + FPN
Faster R-CNN v2	Dos etapas	Sigue el mismo proceso, pero incorpora mejoras en las capas de predicción y en el entrenamiento [16].	ResNet-50 + FPN
Faster R-CNN MobileNet	Dos etapas, versión ligera	Utiliza la misma idea, pero con una red más ligera para reducir el coste computacional y priorizar la velocidad.	MobileNetV3-L + FPN
RetinaNet v1 / v2	Una etapa	Detecta y clasifica objetos directamente en distintas posiciones y escalas. Utiliza la pérdida focal para dar más importancia a los ejemplos difíciles frente al gran número de regiones de fondo [9]. La v2 incorpora mejoras en la cabeza y el entrenamiento.	ResNet-50 + FPN
SSD300	Una etapa	Detecta objetos de distintos tamaños en una imagen de entrada de 300 × 300 píxeles [10].	VGG16
SSDlite320	Una etapa, versión ligera	Utiliza convoluciones más eficientes para reducir el coste computacional y procesa imágenes de 320 × 320 píxeles [17].	MobileNetV3-L

Tabla 12. Los siete modelos comparados.

9. Hiperparámetros

Parámetro	Valor aplicado

Épocas y tamaño (lote)	12 épocas (30 al inicio; 15 con lr 0,0005); lote efectivo de 8 imágenes. A 1024 píxeles, microlotes de 4 × 2.
Semilla aleatoria	0
Optimizador	SGD, con momentum de 0,9 y regularización (weight decay) de 0,0005.
Tasa de aprendizaje	0,0001 en los dos primeros experimentos; 0,001 en el resto y en las arquitecturas; 0,0005 y 0,002 en variantes.
Ajuste de la tasa	Reduce la tasa ×0,1 tras 5 épocas sin mejorar la pérdida de validación; mínimo 1e−10.
Resolución (entrada)	800/1333 píxeles; 1024/1820 en el experimento de resolución.
Datos de entrenamiento	Imágenes originales y tres copias aumentadas de cada una, aplicando el mismo método de aumento base.
Selección del modelo	Se conserva el checkpoint con mayor AP@0,5 en validación.
Entorno de ejecución	PyTorch 2.4.1, torchvision 0.19.1 y GPU NVIDIA RTX 3060 de 12 GB.

Tabla 13. Configuración común de los experimentos medidos.

La Tabla 13 recoge los hiperparámetros comunes utilizados en los experimentos.

Batch size (tamaño del lote). Un lote pequeño puede producir gradientes más variables, mientras que uno grande requiere más memoria. Durante las pruebas, algunos modelos no cabían en la GPU con un lote de 8 imágenes. Por ello, Faster R-CNN v2 y el experimento a 1024 píxeles utilizan microlotes de 4 imágenes con dos acumulaciones de gradientes, manteniendo un lote efectivo de 8. Se fijó este valor para comparar las arquitecturas; su efecto se midió después con Faster R-CNN v1 (apartado 11.3).

Learning rate (tasa de aprendizaje). Determina el tamaño de los pasos que se dan al actualizar los pesos. Una tasa demasiado alta puede hacer que el entrenamiento se vuelva inestable y empeore los resultados. En las pruebas, con 0,0001 el mejor checkpoint se alcanza en la época 29, mientras que con 0,001 se alcanza en la 5. Con 0,002, el máximo llega en la época 1 o 2, pero después el AP disminuye entre 1,1 y 1,4 puntos. Con 0,0005, el máximo se alcanza entre las épocas 5 y 14. Estos resultados muestran que una tasa mayor no garantiza un mejor modelo. El programador reduce la tasa cuando la pérdida de validación deja de mejorar, aunque en las pruebas no siempre lo hace a tiempo para recuperar el mejor resultado (apartado 11.1).

Épocas. Una época corresponde a una pasada completa por las imágenes de entrenamiento. Los primeros experimentos usaron 30 épocas y los siete modelos, 12 (15 con lr 0,0005). En seis de los siete modelos, el mejor checkpoint está entre las épocas 3 y 7; SSD300, en la 12 (en la 18 con 40 épocas, con el mismo AP@0,5; apartado 11.5). La pérdida de validación toca su mínimo pronto (época 4 con 30 épocas) y después aumenta mientras la de entrenamiento baja: hay sobreajuste, no falta de épocas. Por eso, se selecciona el mejor checkpoint de validación en lugar de asumir que el último es el mejor.

Optimizador y programador. Se utiliza SGD para actualizar los pesos en función de los gradientes calculados durante el entrenamiento. El parámetro momentum (0,9) aprovecha parte de la dirección de las actualizaciones anteriores para suavizar las variaciones, mientras que weight decay (0,0005)

penaliza los pesos excesivamente grandes y ayuda a limitar el sobreajuste. Por su parte, el programador baja la tasa cuando la validación se estanca (Tabla 13). No se probaron otros optimizadores, por lo que no se puede afirmar que SGD sea superior a las alternativas.

10. Métricas

El IoU mide cuánto se solapan la caja predicha y la caja real, dividiendo su intersección entre su unión. Para evaluar las detecciones, se exige un IoU ≥ 0,5 y una confianza ≥ 0,5. Las detecciones se emparejan por confianza, es decir, si una coincide con un robot anotado, es un verdadero positivo (TP); si no tiene pareja, es un falso positivo (FP); y si un robot anotado queda sin detectar, es un falso negativo (FN).

La precisión mide qué proporción de las detecciones son correctas (TP/(TP+FP)), mientras que la recall indica qué proporción de los robots anotados se detecta (TP/(TP+FN)). El F1 combina ambas mediante su media armónica. El umbral de confianza establece la confianza mínima para aceptar una detección, al subirlo, suelen disminuir los FP y aumentar la precisión, pero también pueden aumentar los FN y bajar la exhaustividad. Al reducirlo, ocurre lo contrario. Este es el compromiso entre ambas métricas (apartado 11.6).

La AP (Average Precision) resume la curva precisión-exhaustividad al variar el umbral de confianza y se calcula como el área bajo esa curva. AP@0,5 utiliza un IoU de 0,5, mientras que AP@0,75 exige una mayor precisión. AP 0,50:0,95 promedia los resultados para umbrales de IoU entre 0,50 y 0,95; es la métrica conocida como mAP en COCO. La «m» indica que se promedia entre clases y, al trabajar con una sola clase, la mAP coincide con la AP.

11. Experimentos

Los experimentos se organizaron en tres fases. Primero se ajustaron los hiperparámetros con un único modelo, Faster R-CNN v1 (el modelo de partida que propone el enunciado), para no multiplicar el coste de entrenamiento, se comparó el aumento de datos (sin aumento frente al aumento base), la tasa de aprendizaje (0,0001 y 0,001) y la resolución de entrada (apartados 11.1 y 11.2). Después, con los hiperparámetros ya fijados, se compararon las siete arquitecturas (11.3) y se estudió la tasa de aprendizaje con las tres mejores (11.4). Además, se repitió SSD300 con 40 épocas para comprobar si faltaban épocas (11.5) y, por último, se evaluó cada modelo en test una sola vez (11.6).

11.1. Aumento de datos y tasa de aprendizaje Faster R-CNN v1

Experimento	Mejor época	AP50 historial	AP50 evaluación	AP evaluación
Sin aumento lr 0,0001	20	0,9229	0,9228	0,5809
Aumento base lr 0,0001	29	0,9425	0,9416	0,6081
Aumento base lr 0,001	5	0,9432	0,9433	0,6302
Aumento base lr 0,001, 1024 px	2	0,9464	0,9465	0,6047

Tabla 14. Mejor checkpoint por AP50 en validación.

La Tabla 14 compara los cuatro primeros experimentos. El aumento de datos mejora especialmente la detección de objetos difficult, cuyo AP50 pasa de 0,5518 a 0,6341. Sin embargo, al multiplicar por cuatro las imágenes y actualizaciones, no se puede atribuir toda la mejora únicamente a las

transformaciones. Por otro lado, subir la tasa de aprendizaje de 0,0001 a 0,001 mejora el ajuste de las cajas (AP 0,50:0,95: +2,21 puntos; AP75: +3,14), apenas cambia el AP50 (+0,17) y permite alcanzar el mejor resultado en la época 5, frente a la 29.

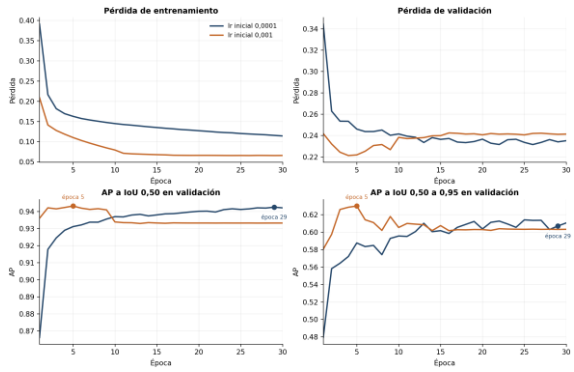


Figura 10. Pérdidas y AP por época. Los puntos señalan los checkpoints elegidos por AP50. Interactiva en el Anexo A.

Con un learning rate de 0,001, la pérdida de validación alcanza su mínimo en la época 4 (0,2212) y después aumenta hasta 0,2413, mientras que la pérdida de entrenamiento sigue bajando hasta 0,0657 (Figura 10). Esto indica un posible sobreajuste, el modelo sigue aprendiendo de las imágenes de entrenamiento, pero empeora en validación. Por ello, se conserva el mejor checkpoint en lugar del último. Como muestra la Figura 20 (Anexo C), el programador reduce la tasa de aprendizaje de 0,001 a 1e−7 en las épocas 11, 17, 23 y 29, pero no consigue recuperar el mejor resultado.

11.2. Resolución de entrada Faster R-CNN v1

Como el rendimiento en objetos pequeños seguía siendo limitado (AP small = 0,1182), se aumentó la resolución de entrada de 800/1333 a 1024/1820 píxeles, manteniendo los pesos, tasa de aprendizaje de 0,001 y la aumentación. Así, un frame pasa de 1333 × 750 a 1820 × 1024 píxeles, lo que supone un 86 % más de píxeles. Debido al consumo de memoria, se utilizaron microlotes de 4 imágenes con acumulación de gradientes en dos pasos.

Mejor por AP50 en validación	800 px	1024 px	Cambio
AP50 todos	0,9433	0,9465	+0,32 pp
AP 0,50:0,95 todos	0,6302	0,6047	−2,55 pp
AP75 todos	0,7460	0,6990	−4,70 pp
AP50 difficult	0,6462	0,6282	−1,80 pp
AP small	0,1182	0,1886	+7,04 pp
FP / FN (conf. 0,5)	190 / 83	324 / 68	+134 / −15
F1 (conf. 0,5)	0,9155	0,8840	−3,15 pp
Latencia por imagen	75,1 ms	121,6 ms	+61,9 %

Tabla 15. Resolución 800 frente a 1024 con el mismo procedimiento.

La Tabla 15 compara ambas resoluciones manteniendo el mismo procedimiento. Con 1024 píxeles mejora la detección de objetos pequeños (AP small pasa de 0,1182 a 0,1886) y se detectan 15 robots más, nueve de ellos difficult. Sin embargo, aumentan los falsos positivos en 134, empeoran el ajuste

de las cajas y el F1 baja de 0,9155 a 0,8840. Además, la latencia aumenta un 61,9 %, pasando de 75,1 a 121,6 ms por imagen.

Aunque el modelo a 1024 píxeles alcanza su mejor AP50 en la época 2 y su máximo AP 0,50:0,95 (0,6322) en la época 5, se mantiene el criterio de selección fijado previamente, basado en AP50. Por tanto, el modelo a 800 píxeles sigue siendo la referencia, ya que el aumento de resolución no mejora el rendimiento de forma uniforme. Por secuencias, el de 800 píxeles mejora el AP50 en 7 de las 11 secuencias de validación, frente a 4 con el de 1024.

11.3. Comparación de arquitecturas

Se entrenaron los siete modelos del apartado 8 con pesos preentrenados de COCO y la cabeza de predicción adaptada a la detección de robots. Para cada modelo se seleccionó el checkpoint con mayor AP@0,5 en validación. La Tabla 16 recoge los resultados y la Figura 11 muestra la evolución del AP por época. Las versiones interactivas de esta y de las demás gráficas del Bloque 2, en las que se puede elegir la métrica, el modelo y el conjunto, se recogen en el Anexo A.

Modelo	Época	AP50 todos	AP50 normales	AP50 difficult	AP 0,5:0,95	F1 máx. (umbral)	ms/imagen	Min
Faster R-CNN v2	3	0,9513	0,9759	0,6422	0,6192	0,9361 (0,95)	99,5	151
RetinaNet v2	6	0,9447	0,9748	0,6213	0,6114	0,9264 (0,65)	67,3	102
Faster R-CNN v1	5	0,9426	0,9733	0,6471	0,6273	0,9305 (0,90)	75,1	106
RetinaNet v1	7	0,9396	0,9687	0,5994	0,6196	0,9229 (0,65)	72,2	98
Faster R-CNN MobileNet	3	0,9281	0,9597	0,6027	0,5789	0,9167 (0,55)	16,0	19
SSD300 VGG16	12	0,9008	0,9404	0,5199	0,5144	0,8926 (0,25)	12,5	19
SSDlite320	7	0,6823	0,7272	0,3233	0,3943	0,6839 (0,20)	4,2	6

Tabla 16. Siete arquitecturas con lr 0,001 en validación (672 imágenes, 1561 robots, 255 difficult). Época del mejor checkpoint, F1 máximo, ms por imagen de una en una y minutos de entrenamiento.

Faster R-CNN v2 obtiene el mejor AP50 (0,9513), seguido de RetinaNet v2 (0,9447), Faster R-CNN v1 (0,9426) y RetinaNet v1 (0,9396). Los cuatro modelos con ResNet-50 presentan diferencias pequeñas, de apenas 1,2 puntos, por lo que no son concluyentes al haberse utilizado una sola semilla. Faster R-CNN alcanza su máximo entre las épocas 3 y 5 y después pierde entre 0,6 y 1 punto de AP mientras la pérdida de entrenamiento sigue bajando, lo que apunta a un sobreajuste temprano. RetinaNet comienza más despacio, pero mantiene un comportamiento más estable. SSD300 todavía mejoraba en la época 12; el apartado 11.5 comprueba si le faltaban épocas.

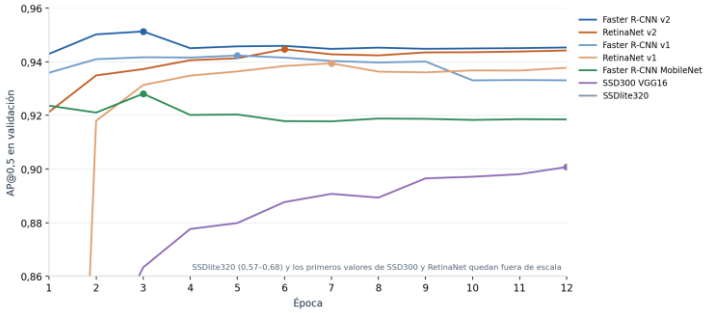


Figura 11. AP@0,5 en validación por época. El punto marca el checkpoint elegido. Interactiva en el Anexo A.

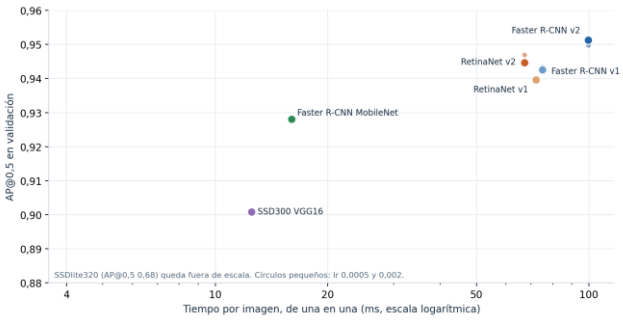


Figura 12. Precisión frente a tiempo por imagen. Interactiva en el Anexo A.

Los modelos ligeros sacrifican precisión a cambio de velocidad. Faster R-CNN MobileNet pierde 2,3 puntos de AP50 respecto a la v2, pero es seis veces más rápido. SSD300 ofrece un AP50 de 0,9008, mientras que SSDlite320 baja a 0,6823, ya que su resolución dificulta detectar robots lejanos.

Los 13 entrenamientos suman 21,8 horas de GPU. Además, los objetos difficult siguen siendo el principal problema, con un AP@0,5 de 0,60–0,65, muy inferior al de los objetos normales. Cambiar de arquitectura no resuelve esta diferencia.

Pruebas sin aumento (Anexo D). A diferencia de lo que propone el enunciado (arquitecturas sin aumento y aumento solo con el mejor modelo), todas se entrenaron con aumento, que ya mejoraba Faster R-CNN v1 (apartado 11.1). Entrenadas también sin aumento durante seis épocas (Tabla 21), Faster R-CNN v2 sigue siendo la mejor (AP@0,5 de 0,9382). Con Faster R-CNN v1 sin aumento y tres épocas se midieron además el lote y la tasa (Tablas 22 y 23). Con lotes de 1 y 2 imágenes el AP@0,5 (0,9332 y 0,9358) iguala al de 8 (0,9326) con menos memoria, y con 16 y 32 la GPU de 12 GB se queda sin memoria (CUDA out of memory). Con la tasa 1e−5 el modelo apenas aprende (AP@0,5 de 0,026), con 0,01 llega al máximo en la época 2 (0,9353) y baja después, y con 0,1 la pérdida es NaN desde la primera época: diverge.

11.4. Tasa de aprendizaje y perfil de aumento

Modelo	lr	Mejor época	AP50 todos	AP50 difficult	AP 0,50:0,95	Min
Faster R-CNN v2	0,0005	5	0,9507	0,6361	0,6126	189
Faster R-CNN v2	0,0010	3	0,9513	0,6422	0,6192	151
Faster R-CNN v2	0,0020	1	0,9497	0,6223	0,6403	151
RetinaNet v2	0,0005	11	0,9451	0,6133	0,6170	126
RetinaNet v2	0,0010	6	0,9447	0,6213	0,6114	102
RetinaNet v2	0,0020	6	0,9469	0,6246	0,6177	101
Faster R-CNN v1	0,0005	14	0,9420	0,6323	0,6104	132
Faster R-CNN v1	0,0010	5	0,9426	0,6471	0,6273	106
Faster R-CNN v1	0,0020	2	0,9434	0,6465	0,6024	105

Tabla 17. Variación de la tasa de aprendizaje en los tres modelos con mejor AP@0,5.

La tasa de aprendizaje apenas cambia el AP@0,5 (Tabla 17), pues las diferencias son inferiores a 0,22 puntos, así que pueden deberse a la semilla. Se mantiene 0,001 porque logra resultados similares a

0,0005 en menos épocas. Con 0,002, Faster R-CNN alcanza su máximo en una o dos épocas y después pierde entre 1,1 y 1,4 puntos, con la pérdida de validación subiendo (de 0,182 a 0,224 en la v2): inestabilidad y sobreajuste, no infraajuste.

También se comparó con los cambios de aumentación de datos para la robustez, con sombras, llovizna y más imágenes difficult. Los resultados son prácticamente iguales: AP@0,5 de 0,9426 frente a 0,9433. No se puede confirmar una mejora y se mantiene el aumento base.

11.5. Comprobación con más épocas

Para comprobar si 12 épocas eran pocas, se repitió con 40 SSD300, el único modelo cuyo mejor checkpoint caía en la última época, sin cambiar nada más (Figura 13). Con la misma semilla, sus 12 primeras épocas coinciden con el entrenamiento original.

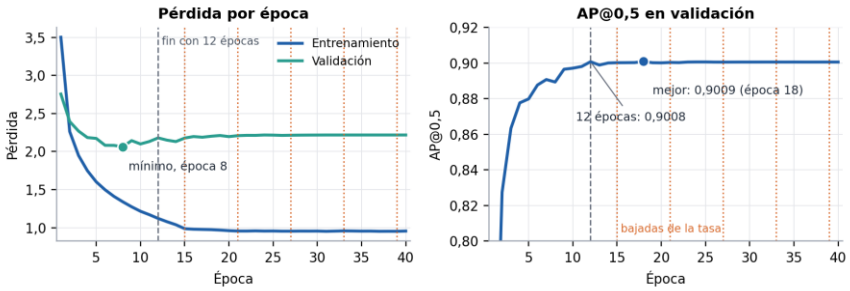


Figura 13. SSD300 con 40 épocas. La línea discontinua marca el final del entrenamiento de 12 épocas y las punteadas, las bajadas de la tasa. Interactiva en el Anexo A.

Las 28 épocas adicionales bajan la pérdida de entrenamiento un 15 % (de 1,121 a 0,955), pero el AP@0,5 en validación solo pasa de 0,9008 a 0,9009 (época 18) y termina en 0,9006. La pérdida de validación tiene su mínimo en la época 8 y después sube, pese a las bajadas de la tasa: es el mismo sobreajuste que en Faster R-CNN (apartado 11.1), no falta de épocas. Solo mejora algo el ajuste de las cajas (AP 0,50:0,95 de 0,514 a 0,536); los difficult no (0,5145 frente a 0,5199). Por su coste, 62 minutos frente a 19, no se repitió con el resto de modelos, cuyo mejor checkpoint está entre las épocas 3 y 7.

11.6. Evaluación final en test

Se elige el modelo con mayor AP@0,5 en validación y el umbral con mejor F1.

Modelo	AP50 val.	AP50 test	Diff. test	Umbral	Precis.	Exhaust.	F1	FP / FN
Faster R-CNN v2	0,9513	0,9523	0,6921	0,95	0,9447	0,9332	0,9389	94 / 115
RetinaNet v2	0,9447	0,9457	0,6313	0,65	0,9409	0,9064	0,9233	98 / 161
Faster R-CNN v1	0,9426	0,9355	0,6057	0,90	0,9224	0,9250	0,9237	134 / 129
RetinaNet v1	0,9396	0,9362	0,6273	0,65	0,9226	0,9210	0,9218	133 / 136
Faster R-CNN MobileNet	0,9281	0,9240	0,5744	0,55	0,9145	0,9140	0,9142	147 / 148
SSD300 VGG16	0,9008	0,8797	0,4316	0,25	0,9128	0,8216	0,8648	135 / 307
SSDlite320	0,6823	0,6412	0,2742	0,20	0,7284	0,5985	0,6571	384 / 691

Tabla 18. Evaluación final en test, una vez por modelo y con el umbral fijado en validación.

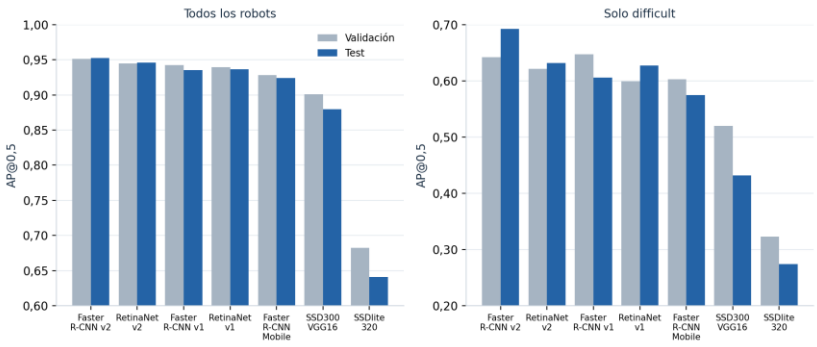


Figura 14. AP@0,5 en validación y en test. Interactiva en el Anexo A.

Faster R-CNN v2 con lr 0,001 es el modelo final y alcanza un AP@0,5 de 0,9523 en test (0,9513 en validación). El AP es de 0,9658 en robots normales y 0,6921 en difficult. Con umbral 0,95 detecta 1606 de los 1721 robots, con una exhaustividad de 0,9332, precisión de 0,9447 y F1 de 0,9389. El mejor F1 en test sería 0,9413 con umbral 0,90, una diferencia mínima de 0,24 puntos. En general, los resultados de test coinciden con los de validación: las diferencias son pequeñas en los modelos ResNet-50, mientras que las SSD pierden más precisión (2,1 y 4,1 puntos).

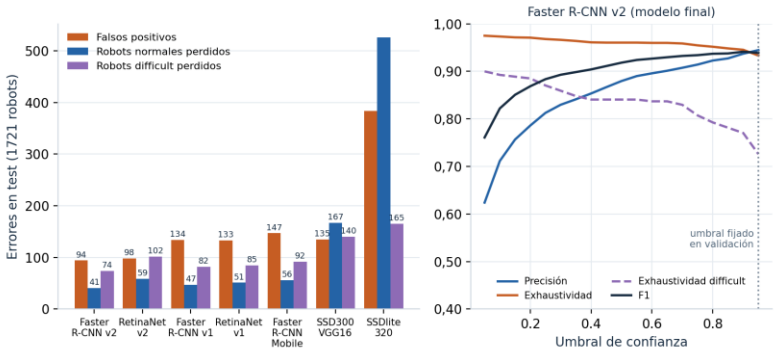


Figura 15. Errores en test con el umbral fijado en validación (izquierda) y curvas de umbral del modelo final (derecha). Interactivas en el Anexo A: izquierda y derecha.

Como se ve en la Figura 15, 74 de los 115 robots que no detecta el modelo final (64 %) son difficult, aunque representan solo el 16 % del total. Subir el umbral de 0,5 a 0,95 reduce los falsos positivos de 226 a 94, pero también baja la exhaustividad en difficult de 0,84 a 0,73.

Para la demo, Faster R-CNN MobileNet es una alternativa más rápida que obtiene un AP@0,5 de 0,9240 y un F1 de 0,9143, con 16 ms por imagen frente a 96,3 ms del modelo final.

12. Análisis cualitativo

La Figura 16 muestra dos predicciones de Faster R-CNN v1 (lr 0,001) sobre validación. En la primera hay un acierto completo, con tres detecciones correctas, incluida una con confianza 0,51 sobre un robot difficult. Las cajas verdes son aciertos y las rojas, detecciones sin pareja. En la segunda, de GR02_0001 de 2024, se confirmó que faltaban etiquetas (Figura 17, izquierda), por lo que algunos falsos positivos no son errores reales del detector, sino que el conjunto de validación no estaba bien etiquetado.



Figura 16. Predicciones de Faster R-CNN v1 (lr 0,001) sobre validación, acierto en PIDS2025_GR01_0010/frame_1566 (TP 3, FP 0, FN 0) y el caso con más errores, PIDS2024_GR02_0001/frame_0492 (TP 2, FP 4, FN 0).

Al revisar las predicciones, se distinguen errores del detector y etiquetas incompletas. En la Figura 17, a la izquierda, aparecen cuatro robots, pero el XML solo incluye una caja, por lo que detectar los otros tres cuenta como FP. Las etiquetas se mantienen para conservar la comparación. Esto limita la interpretación de los 190 FP del modelo a 800 píxeles y los 324 del de 1024, aunque no se sabe cuántos se deben a anotaciones omitidas.



Figura 17. Izquierda: caso confirmado de anotación incompleta; la caja azul es la única del XML. Derecha: modelo a 1024 píxeles en PIDS2025_GR03_0011/frame_0600, con TP 4, FP 4 y FN 1 frente a las etiquetas.

Los fallos reales se concentran en tres casos. Primero, los objetos difficult, que acumulan 65 de los 83 FN de Faster R-CNN v1 (lr 0,001) y 74 de los 115 del modelo final en test, aunque representan solo el 26,4 % de las cajas anotadas (375 de 1420). Segundo, los robots lejanos, con un AP small de 0,12–0,19, que son difíciles de detectar y están poco representados: no hay cajas originales de menos de 30 píxeles de ancho. Tercero, algunas secuencias especialmente difíciles: en test, PIDS2025_GR02_0009 pierde 40 de 204 robots y PIDS2025_GR04_0000 pierde 10 de 21; en validación, GR04_0003 de 2024 pierde sus seis objetos difficult. También hay falsos positivos sobre una pierna y cajas solapadas alrededor de un robot (Figura 17, derecha), que no se explican por etiquetas omitidas.

12.1. Conclusiones y trabajo pendiente

El modelo final elegido es Faster R-CNN v2 con lr 0,001, cuyo checkpoint de la época 3 alcanza un AP@0,5 de 0,9523 en test (0,9658 en robots normales y 0,6921 en difficult) y un F1 de 0,9389 con el umbral 0,95 fijado en validación. Se prefiere a Faster R-CNN MobileNet, seis veces más rápido, por su mayor precisión, aunque habrá que comprobar en el robot si su latencia es adecuada.

La arquitectura influye más que la tasa de aprendizaje o el aumento de datos y los difficult siguen siendo el principal problema, las SSD tienen dificultades con robots lejanos y Faster R-CNN sobreajusta pronto, por lo que el checkpoint se selecciona mediante validación.

Referencias

[1] labelImg. Herramienta de anotación PascalVOC. https://github.com/HumanSignal/labelImg

[2] Albumentations 1.4.18. Código de transformaciones. https://github.com/albumentations-team/albumentations/tree/1.4.18

[3] S. Ren et al. Faster R-CNN Towards Real-Time Object Detection with Region Proposal Networks. 2015. https://arxiv.org/abs/1506.01497

[4] torchvision 0.19. Faster R-CNN ResNet-50 FPN. https://docs.pytorch.org/vision/0.19/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html

[5] Repositorio del grupo. Datos, criterios, notebooks y resultados. https://github.com/javiersaguar/PIDS-Proyecto2

[6] Web del proyecto interactiva. https://javiersaguar.github.io/PIDS-Proyecto2/

[7] Notebook del EDA ejecutado (eda.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/eda.html

[8] Notebook del aumento de datos ejecutado (data_aug.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/data_aug.html

[9] T.-Y. Lin et al. Focal Loss for Dense Object Detection. 2017. https://arxiv.org/abs/1708.02002

[10] W. Liu et al. SSD: Single Shot MultiBox Detector. 2016. https://arxiv.org/abs/1512.02325

[11] K. He et al. Deep Residual Learning for Image Recognition. 2016. https://arxiv.org/abs/1512.03385

[12] T.-Y. Lin et al. Feature Pyramid Networks for Object Detection. 2017. https://arxiv.org/abs/1612.03144

[13] A. Howard et al. Searching for MobileNetV3. 2019. https://arxiv.org/abs/1905.02244

[14] K. Simonyan y A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. 2015. https://arxiv.org/abs/1409.1556

[15] T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. 2014. https://arxiv.org/abs/1405.0312

[16] Y. Li et al. Benchmarking Detection Transfer Learning with Vision Transformers. 2021. https://arxiv.org/abs/2111.11429

[17] M. Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018. https://arxiv.org/abs/1801.04381

[18] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. https://arxiv.org/abs/1912.01703

[19] DJI. RoboMaster EP. https://www.dji.com/robomaster-ep

[20] M. Everingham et al. The PASCAL Visual Object Classes (VOC) Challenge. International Journal of Computer Vision, 88, 303–338. 2010. https://doi.org/10.1007/s11263-009-0275-4

Anexos

Anexo A. Figuras interactivas

La web del proyecto [6] reúne las versiones interactivas de las figuras de los dos bloques: permiten elegir vídeo, modelo, métrica o conjunto y consultar cada valor. La Tabla 19 relaciona cada figura o tabla del informe con su gráfica interactiva.

Figura o tabla	Gráfica interactiva	Enlace
Figura 6	Centros de las cajas de cada vídeo, todos y difficult	javiersaguar.github.io/PIDS-Proyecto2/#distribucion
Figura 6	Evolución de los centros en ventanas de 5 segundos	javiersaguar.github.io/PIDS-Proyecto2/#evolucion
Figura 7	Robots por fotograma en los diez vídeos	javiersaguar.github.io/PIDS-Proyecto2/#recuentos
Figura 10	Pérdidas y AP por época de los primeros experimentos	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-epocas
Tablas 14 y 15	Comparación de los primeros experimentos por métrica	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-comparacion
Apartado 11.2	AP de normales y difficult en cada experimento	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-grupos
Apartado 11.2	AP por secuencia de validación	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-secuencias
Figura 11	AP por época de cada arquitectura	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#arquitecturas
Figura 13	Pérdida, AP y tasa de cada entrenamiento, con SSD300 a 40 épocas	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#aprendizaje
Tabla 17	Efecto de la tasa de aprendizaje	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#tasa
Tabla 16	Tiempo de entrenamiento y época del mejor checkpoint	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#coste
Figura 12	Precisión frente a tiempo por imagen	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#latencia
Figura 14	AP@0,5 en validación y en test	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#validacion_test
Figura 15	Errores en test de cada modelo	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#errores

Figura o tabla	Gráfica interactiva	Enlace
Figura 15	Precisión, exhaustividad y F1 según el umbral de confianza	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#umbral
Tabla 18	AP por secuencia de test	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#secuencias
Apartado 11	Todas las medidas de los 13 entrenamientos en una matriz	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#mapa

Tabla 19. Gráficas interactivas de la web del proyecto y su relación con el informe.

Anexo B. Transformaciones descartadas

Transformación	Motivo del descarte
Volteo vertical	El robot nunca está boca abajo y el suelo nunca aparece arriba.
Giros de 90°	La escena tiene una orientación fija; el robot no aparece de lado.
Perspectiva	En Albumentations 1.4.18 la caja devuelta no coincide con el robot (IoU de 0 en la prueba automática). Introduciría cajas falsas en el entrenamiento.
Recorte 640 × 480 redimensionado a 1280 × 720	Es el aumento de la asignatura. Pasa de 4:3 a 16:9 y ensancha los robots un 33 %; sobre nuestras imágenes pierde además el 14,1 % de los robots. Se sustituye por la escala sin deformar.
Desenfoque uniforme de 7 px al 50 %	El 27 % de los robots ya es difficult, casi siempre por movimiento. Desenfocar la mitad de las copias haría el entrenamiento más borroso que la realidad, y el desenfoque uniforme no se parece al de movimiento.
Intercambio de canales	Produce colores que la cámara nunca dará.
Niebla, lluvia, destellos y sombras (perfil base)	No son condiciones de la demo, que es en interior. Sombras y lluvia se exploran en perfiles separados (apartado 6.1).
Mosaico y copy-paste	Son útiles con muchos objetos pequeños; aquí hay 1,8 robots por imagen y la escala ya aporta robots más pequeños.

Tabla 20. Transformaciones descartadas y motivo.

Anexo C. Figuras complementarias

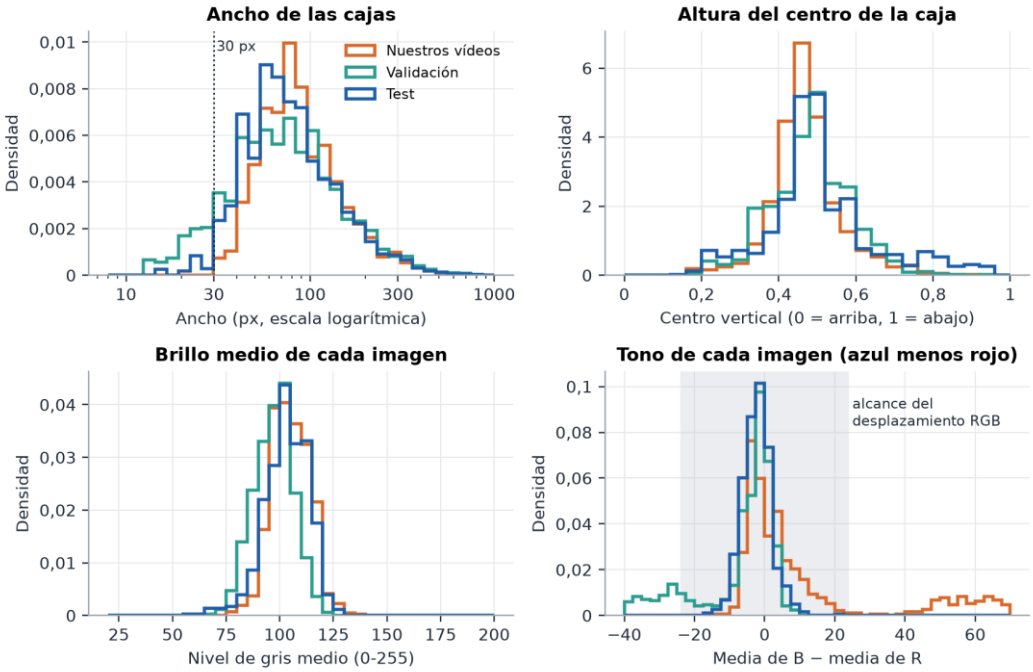


Figura 18. Distribución del ancho de las cajas, la altura de su centro, el brillo y el tono de cada imagen en nuestros vídeos, validación y test. La franja gris es el alcance del desplazamiento RGB del aumento (±12 por canal, hasta ±24 en B − R).



Figura 19. Una imagen de V06 y siete transformaciones del aumento por separado, con las cajas transformadas.

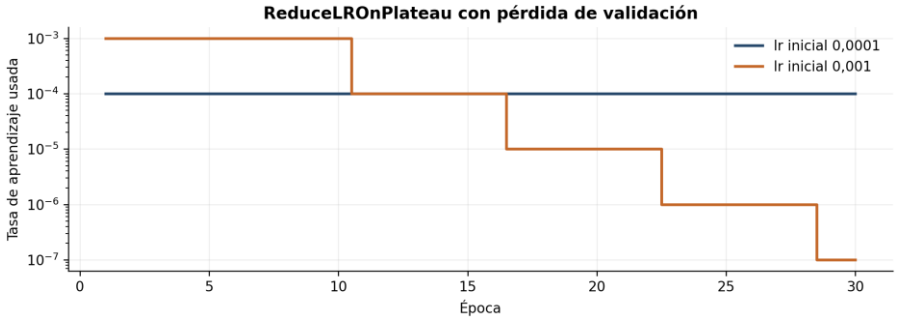


Figura 20. Tasa usada en cada época, y bajadas del programador.

La Figura 21 amplía el apartado 4.2 con análisis del notebook del EDA [7]. La nitidez (varianza del Laplaciano de la imagen en gris) depende del escenario: las medianas de los exteriores V04 y V09 son 328 y 278, y las de los interiores van de 43 a 130, porque con mucha luz la cámara usa exposiciones

cortas. Dentro de cada vídeo, cuanto menos nítida es una imagen, más cajas difficult tiene: el 55 % en el cuarto menos nítido (cuartiles de la nitidez de cada imagen respecto a la mediana de su vídeo, sobre las 767 imágenes con robots) frente al 11 % en el más nítido, y el 58 % de las imágenes con algún difficult frente al 21 %. Entre vídeos la relación es débil (V05 es de los menos nítidos y solo tiene un 10,6 % de difficult), por eso se mide dentro de cada vídeo. Como el criterio de anotación marca como difficult los robots movidos, esto es coherente con incluir desenfoque de movimiento en el aumento (apartado 5.3) y con que los difficult sean el principal error del Bloque 2. Por último, nunca hay más de tres robots por imagen y el 62 % tiene al menos dos.

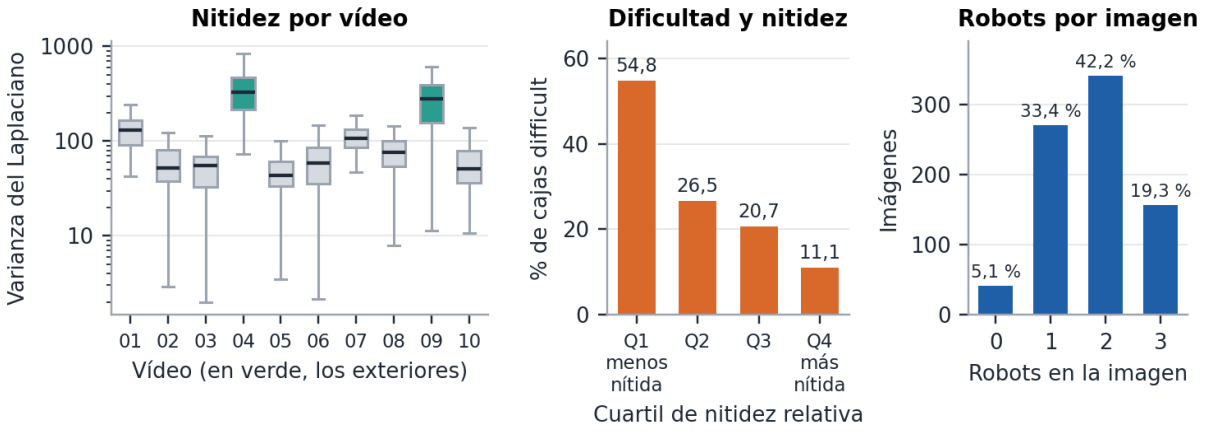


Figura 21. Izquierda: nitidez de las imágenes de cada vídeo, en escala logarítmica; en verde, los exteriores. Centro: porcentaje de cajas difficult según el cuartil de nitidez relativa de la imagen dentro de su vídeo. Derecha: robots por imagen, de las 808 imágenes.

Anexo D. Pruebas cortas: orden de los experimentos, lote y tasa de aprendizaje

Estas pruebas se hicieron al final para medir lo que no estaba en las fases anteriores. Todas se evalúan en validación, con la semilla 0 y la configuración de la Tabla 13 salvo lo que se indica, y entrenan solo con las 796 imágenes originales, sin aumento (100 actualizaciones por época con lote 8). El lote y la tasa se prueban con Faster R-CNN v1 durante tres épocas; la comparación de arquitecturas, durante seis. Cada resultado sale de una sola ejecución, por lo que diferencias de unas décimas no son concluyentes. RetinaNet y SSD300 mejoran hasta la última época y con ocho veces menos actualizaciones que los entrenamientos con aumento no han convergido, así que su posición en la Tabla 21 no es comparable con la de la Tabla 16.

Modelo	Mejor época (de 6)	AP@0,5 sin aumento	AP 0,50:0,95 sin aumento	AP@0,5 con aumento (12 épocas)	Minutos
Faster R-CNN v2	6	0,9382	0,6118	0,9513	18,8
Faster R-CNN v1	5	0,9347	0,5813	0,9426	13,2
RetinaNet v2	6	0,9188	0,5814	0,9447	12,6
Faster R-CNN MobileNet	5	0,9174	0,5768	0,9281	2,5
RetinaNet v1	6	0,8828	0,5727	0,9396	12,2
SSD300 VGG16	6	0,7754	0,4667	0,9008	2,3
SSDlite320	3	0,5975	0,3268	0,6823	0,8

Tabla 21. Siete arquitecturas entrenadas solo con las 796 imágenes originales (lr 0,001, seis épocas, validación). Con mejor época 6, el AP@0,5 aún subía al terminar.

Lote	Pasos por época	AP@0,5 en las épocas 1 / 2 / 3	Mejor AP@0,5 (época)	AP 0,50:0,95	Memoria pico	Min
1	796	0,9329 / 0,9332 / 0,9280	0,9332 (2)	0,6084	1,3 GiB	7,2
2	398	0,9341 / 0,9358 / 0,9285	0,9358 (2)	0,5851	2,3 GiB	6,7
8	100	0,9128 / 0,9287 / 0,9326	0,9326 (3)	0,5886	8,0 GiB	6,6
16	—	Sin memoria (CUDA out of memory)	—	—	—	—
32	—	Sin memoria (CUDA out of memory)	—	—	—	—

Tabla 22. Efecto del tamaño del lote en Faster R-CNN v1 sin aumento (lr 0,001, tres épocas, validación). El lote 8 son las tres primeras épocas de la prueba de seis. Los lotes 16 y 32 no caben en la GPU (11,6 GiB útiles): al fallar tenían asignados 10,48 y 9,47 GiB y pedían 1008 MiB más.

Tasa	AP@0,5 en las épocas 1 / 2 / 3	Mejor AP@0,5 (época)	Pérdida de entrenamiento (época 3)	Resultado
1e−5	0,0034 / 0,0089 / 0,0261	0,0261 (3)	0,461	Aprende demasiado despacio
0,001	0,9128 / 0,9287 / 0,9326	0,9326 (3)	0,145	Referencia: sigue mejorando hasta la época 5
0,01	0,9337 / 0,9353 / 0,9308	0,9353 (2)	0,107	Llega antes al máximo y baja después
0,1	0,0000 / 0,0000 / 0,0000	0,0000 (1)	NaN	Diverge desde la primera época

Tabla 23. Efecto de la tasa de aprendizaje en Faster R-CNN v1 sin aumento (lote 8, tres épocas, validación).

Respuesta a la revisión anterior

La Tabla 24 recoge todos los comentarios de la revisión de la entrega 1 y dónde se ha incorporado cada corrección. Los números de figura, tabla y apartado son los de esta versión.

#	Comentario recibido	Sección	Cambio realizado
1	C1.1. Añadir referencia o breve explicación de cómo es el robot (opcionalmente, con una imagen).	1.1	Se describe el RoboMaster EP y sus partes, con una imagen de nuestros vídeos (Figura 1) y la referencia del fabricante [19].
2	C1.2. ¿Por qué 808? (No vale decir que es porque lo han pedido los profesores).	2.1	Se justifica el intervalo de 24 fotogramas midiendo la redundancia con intervalos de 6 a 96 (Tabla 3). Las 808 imágenes son la consecuencia de ese intervalo y de la duración grabada, no un objetivo.
3	En un informe técnico no se habla de variables de código ni de archivos; eliminar esas menciones.	2.1, 5, 7, 9, 11 y 12	Se sustituyen variables, funciones, ficheros y nombres internos de experimentos por su descripción. Los ficheros solo aparecen en las referencias y en los anexos.
4	C1.3. Añadir referencia a PascalVOC.	3	Se cita el artículo del PASCAL VOC Challenge [20].
5	C1.4. Pocas reflexiones sobre cómo los resultados del EDA afectan a los modelos.	4.1 a 4.6	Cada análisis termina con su consecuencia para el entrenamiento, y el nuevo apartado 4.6 (Tabla 7) las contrasta con los resultados del Bloque 2.
6	Buen trabajo con la web interactiva; moverla a los anexos.	4.3, 11 y Anexo A	Los enlaces salen de los pies de figura y se reúnen en el Anexo A, con una entrada por gráfica de los dos bloques.
7	C1.5. Se analiza la posición de los centros. ¿Qué pasa con el ancho y el alto de las cajas?	4.4	Nuevo análisis de ancho y alto frente a la posición (Figura 8): el tamaño depende de la distancia (correlación de 0,68) y las cajas son casi cuadradas en todo el rango.
8	C1.6. Cinco análisis para cinco personas; aplicar un par de técnicas adicionales.	4.4 y 4.5	Se añaden tres: tamaño frente a posición (perspectiva), solapamiento entre robots frente a la NMS y comparación de nuestros vídeos con validación y test (Tabla 6 y Figura 18, en el Anexo C).
9	Transformaciones descartadas: si falta espacio, se puede mover a los anexos.	5.5 y Anexo B	La tabla pasa al Anexo B; en el cuerpo queda un resumen de los descartes principales.
10	C1.7. En algunas técnicas solo aparece el rango, sin probabilidad. Arreglarlo en todas las tablas.	5.1 a 5.3	Las tablas separan rango y probabilidad. En las transformaciones que se sortean juntas se da la probabilidad de cada una y la del grupo.
11	C1.8. Justificar el desplazamiento RGB.	4.5 y 5.2	Una secuencia de validación tiene un tono rojizo (B − R = −28) que no aparece en nuestros vídeos: es un balance de blancos distinto, que es lo que simula esta transformación.

#	Comentario recibido	Sección	Cambio realizado
12	Opcional: mostrar una imagen de ejemplo con cada transformación aplicada por separado.	5.5 y Anexo C	La Figura 19 (Anexo C) muestra una misma imagen con siete transformaciones por separado.
13	Revisión propia del grupo.	2, 4.2, 5.4, 9 y 11	Se corrigen el nivel del título 2, que no salía en el índice, y una referencia al apartado 5.6, que no existía. Se añade la comprobación con 40 épocas (apartado 11.5).

Tabla 24. Trazabilidad de las correcciones respecto a la entrega 1.